lvgl_pc_minimal

Generated by Doxygen 1.16.1

1 Directory Hierarchy 1
1.1 Directories . . . 1

2 Data Structure Index 3
2.1 Data Structures . . . 3

3 File Index 5
3.1 File List . . . 5

4 Directory Documentation 7
4.1 display_backends Directory Reference . . . 7
4.2 indev_backends Directory Reference . . . 7
4.3 lib Directory Reference . . . 7
4.4 src Directory Reference . . . 8

5 Data Structure Documentation 9
5.1 app_snapshot_t Struct Reference . . . 9
5.1.1 Detailed Description . . . 9
5.1.2 Field Documentation . . . 9
5.1.2.1 connected . . . 9
5.1.2.2 heartbeat . . . 9
5.1.2.3 myvar1 . . . 10
5.2 backend_handle_t Union Reference . . . 10
5.2.1 Detailed Description . . . 10
5.2.2 Field Documentation . . . 10
5.2.2.1 display . . . 10
5.2.2.2 indev . . . 10
5.3 backend_t Struct Reference . . . 11
5.3.1 Detailed Description . . . 11
5.3.2 Field Documentation . . . 11
5.3.2.1 handle . . . 11
5.3.2.2 name . . . 11
5.3.2.3 type . . . 11
5.4 display_backend_t Struct Reference . . . 11
5.4.1 Detailed Description . . . 12
5.4.2 Field Documentation . . . 12
5.4.2.1 display . . . 12
5.4.2.2 init_display . . . 12
5.4.2.3 run_loop . . . 12
5.5 indev_backend_t Struct Reference . . . 12
5.5.1 Detailed Description . . . 13
5.5.2 Field Documentation . . . 13
5.5.2.1 init_indev . . . 13
5.6 simulator_settings_t Struct Reference . . . 13
5.6.1 Detailed Description . . . 13

5.6.2 Field Documentation . . . 13
5.6.2.1 fullscreen . . . 13
5.6.2.2 maximize . . . 13
5.6.2.3 rotation . . . 14
5.6.2.4 window_height . . . 14
5.6.2.5 window_width . . . 14

6 File Documentation 15
6.1 app_runtime.cpp File Reference . . . 15
6.1.1 Function Documentation . . . 15
6.1.1.1 app_runtime_get_snapshot() . . . 15
6.1.1.2 app_runtime_init() . . . 15
6.1.1.3 app_runtime_start() . . . 16
6.1.1.4 app_runtime_stop() . . . 16
6.2 app_runtime.cpp . . . 16
6.3 app_runtime.h File Reference . . . 17
6.3.1 Function Documentation . . . 18
6.3.1.1 app_runtime_get_snapshot() . . . 18
6.3.1.2 app_runtime_init() . . . 18
6.3.1.3 app_runtime_start() . . . 18
6.3.1.4 app_runtime_stop() . . . 18
6.4 app_runtime.h . . . 18
6.5 backends.h File Reference . . . 19
6.5.1 Detailed Description . . . 19
6.5.2 Typedef Documentation . . . 19
6.5.2.1 backend_init_t . . . 19
6.5.2.2 display_init_t . . . 20
6.5.2.3 indev_init_t . . . 20
6.5.2.4 run_loop_t . . . 20
6.5.3 Enumeration Type Documentation . . . 20
6.5.3.1 backend_type_t . . . 20
6.5.4 Function Documentation . . . 20
6.5.4.1 backend_init_drm() . . . 20
6.5.4.2 backend_init_evdev() . . . 20
6.5.4.3 backend_init_fbdev() . . . 21
6.5.4.4 backend_init_glfw3() . . . 21
6.5.4.5 backend_init_sdl() . . . 21
6.5.4.6 backend_init_wayland() . . . 21
6.5.4.7 backend_init_x11() . . . 21
6.6 backends.h . . . 21
6.7 drm.c File Reference . . . 22
6.7.1 Detailed Description . . . 22
6.8 drm.c . . . 23

6.9 fbdev.c File Reference . . . 24
6.9.1 Detailed Description . . . 24
6.10 fbdev.c . . . 24
6.11 glfw3.c File Reference . . . 25
6.11.1 Detailed Description . . . 26
6.12 glfw3.c . . . 26
6.13 sdl.c File Reference . . . 27
6.13.1 Detailed Description . . . 28
6.14 sdl.c . . . 28
6.15 wayland.c File Reference . . . 29
6.15.1 Detailed Description . . . 29
6.16 wayland.c . . . 30
6.17 x11.c File Reference . . . 31
6.17.1 Detailed Description . . . 31
6.18 x11.c . . . 32
6.19 driver_backends.c File Reference . . . 33
6.19.1 Detailed Description . . . 34
6.19.2 Function Documentation . . . 34
6.19.2.1 driver_backends_init_backend() . . . 34
6.19.2.2 driver_backends_is_supported() . . . 34
6.19.2.3 driver_backends_print_supported() . . . 35
6.19.2.4 driver_backends_register() . . . 35
6.19.2.5 driver_backends_run_loop() . . . 35
6.19.3 Variable Documentation . . . 36
6.19.3.1 available_backends . . . 36
6.19.3.2 backends . . . 36
6.19.3.3 sel_display_backend . . . 36
6.19.3.4 settings . . . 36
6.20 driver_backends.c . . . 37
6.21 driver_backends.h File Reference . . . 40
6.21.1 Detailed Description . . . 40
6.21.2 Function Documentation . . . 41
6.21.2.1 driver_backends_init_backend() . . . 41
6.21.2.2 driver_backends_is_supported() . . . 41
6.21.2.3 driver_backends_print_supported() . . . 42
6.21.2.4 driver_backends_register() . . . 42
6.21.2.5 driver_backends_run_loop() . . . 42
6.22 driver_backends.h . . . 43
6.23 evdev.c File Reference . . . 43
6.23.1 Detailed Description . . . 43
6.24 evdev.c . . . 44
6.25 mouse_cursor_icon.c File Reference . . . 46
6.25.1 Detailed Description . . . 46

6.25.2 Variable Documentation . . . 46
6.25.2.1 mouse_cursor_icon . . . 46
6.25.2.2 mouse_cursor_icon_map . . . 47
6.26 mouse_cursor_icon.c . . . 47
6.27 simulator_settings.h File Reference . . . 48
6.27.1 Detailed Description . . . 49
6.28 simulator_settings.h . . . 49
6.29 simulator_util.c File Reference . . . 49
6.29.1 Detailed Description . . . 50
6.29.2 Function Documentation . . . 50
6.29.2.1 die() . . . 50
6.29.2.2 getenv_default() . . . 50
6.30 simulator_util.c . . . 51
6.31 simulator_util.h File Reference . . . 52
6.31.1 Detailed Description . . . 52
6.31.2 Function Documentation . . . 52
6.31.2.1 die() . . . 52
6.31.2.2 getenv_default() . . . 52
6.32 simulator_util.h . . . 53
6.33 main.c File Reference . . . 53
6.33.1 Macro Definition Documentation . . . 54
6.33.1.1 DEFAULT_DISPLAY_H . . . 54
6.33.1.2 DEFAULT_DISPLAY_W . . . 55
6.33.2 Function Documentation . . . 55
6.33.2.1 action_my_action() . . . 55
6.33.2.2 configure_simulator() . . . 55
6.33.2.3 lv_ui_example() . . . 55
6.33.2.4 main() . . . 55
6.33.2.5 print_lvgl_version() . . . 56
6.33.2.6 print_usage() . . . 56
6.33.2.7 ui_runtime_sync_timer_cb() . . . 56
6.33.2.8 ui_tick_timer_cb() . . . 57
6.33.3 Variable Documentation . . . 57
6.33.3.1 selected_backend . . . 57
6.33.3.2 settings . . . 57
6.34 main.c . . . 57
6.35 ui_example.c File Reference . . . 60
6.35.1 Function Documentation . . . 60
6.35.1.1 lv_ui_example() . . . 60
6.35.1.2 value_changed_event_cb() . . . 60
6.36 ui_example.c . . . 61
6.37 ui_logic.c File Reference . . . 61
6.37.1 Function Documentation . . . 61

6.37.1.1 get_var_myvar1() 61
6.37.1.2 set_var_myvar1() 62
6.37.2 Variable Documentation 62
6.37.2.1 myvar1 62
6.38 ui_logic.c 62

Chapter 1

Directory Hierarchy

1.1 Directories

- display_backends . . . 7
  - drm.c . . . 22
  - fbdev.c . . . 24
  - glfw3.c . . . 25
  - sdl.c . . . 27
  - wayland.c . . . 29
  - x11.c . . . 31
- indev_backends . . . 7
  - evdev.c . . . 43
- lib . . . 7
  - display_backends . . . 7
    - drm.c . . . 22
    - fbdev.c . . . 24
    - glfw3.c . . . 25
    - sdl.c . . . 27
    - wayland.c . . . 29
    - x11.c . . . 31
  - indev_backends . . . 7
    - evdev.c . . . 43
  - backends.h . . . 19
  - driver_backends.c . . . 33
  - driver_backends.h . . . 40
  - mouse_cursor_icon.c . . . 46
  - simulator_settings.h . . . 48
  - simulator_util.c . . . 49
  - simulator_util.h . . . 52
- src . . . 8
  - lib . . . 7
    - display_backends . . . 7
      - drm.c . . . 22
      - fbdev.c . . . 24
      - glfw3.c . . . 25
      - sdl.c . . . 27
      - wayland.c . . . 29
      - x11.c . . . 31
    - indev_backends . . . 7

evdev.c . . . 43
backends.h . . . 19
driver_backends.c . . . 33
driver_backends.h . . . 40
mouse_cursor_icon.c . . . 46
simulator_settings.h . . . 48
simulator_util.c . . . 49
simulator_util.h . . . 52
app_runtime.cpp . . . 15
app_runtime.h . . . 17
main.c . . . 53
ui_example.c . . . 60
ui_logic.c . . . 61

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

app_snapshot_t 9
backend_handle_t 10
backend_t 11
display_backend_t 11
indev_backend_t 12
simulator_settings_t 13

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

app_runtime.cpp 15
app_runtime.h 17
backends.h
Interface for abstraction layer of a device backend 19
drm.c
The DRM/KMS backend 22
fbdev.c
Legacy framebuffer device 24
glfw3.c
The GLFW3 backend 25
sdl.c
The backend for the SDL simulator 27
wayland.c
The wayland backend 29
x11.c
The backend for the X11 windowing system 31
driver_backends.c
Copyright (c) 2025 EDGEMTech Ltd 33
driver_backends.h
Abstraction to support multiple graphical driver backends at the same time without recompiling everything each time 40
evdev.c
The lib evdev driver 43
mouse_cursor_icon.c
The C array containing the mouse cursor icon 46
simulator_settings.h
Global simulator settings 48
simulator_util.c
Utility functions Copyright (c) 2025 EDGEMTech Ltd 49
simulator_util.h
Simulator_util.h - Header file for the utility functions used by the simulator 52
main.c 53
ui_example.c 60
ui_logic.c 61

Chapter 4

Directory Documentation

4.1 display_backends Directory Reference

Files

- file drm.c

  The DRM/KMS backend.
- file fbdev.c

  Legacy framebuffer device.
- file glfw3.c

  The GLFW3 backend.
- file sdl.c

  The backend for the SDL simulator.
- file wayland.c

  The wayland backend.
- file x11.c

  The backend for the X11 windowing system.

4.2 indev_backends Directory Reference

Files

- file evdev.c

  The lib evdev driver.

4.3 lib Directory Reference

Directories

- directory display_backends
- directory indev_backends

Files

- file backends.h

  Interface for abstraction layer of a device backend.
- file driver_backends.c

  Copyright (c) 2025 EDGEMTech Ltd.
- file driver_backends.h

  provides an abstraction to support multiple graphical driver backends at the same time without recompiling everything each time
- file mouse_cursor_icon.c

  The C array containing the mouse cursor icon.
- file simulator_settings.h

  global simulator settings
- file simulator_util.c

  Utility functions Copyright (c) 2025 EDGEMTech Ltd.
- file simulator_util.h

  simulator_util.h - Header file for the utility functions used by the simulator

4.4 src Directory Reference

Directories

- directory lib

Files

- file app_runtime.cpp
- file app_runtime.h
- file main.c
- file ui_example.c
- file ui_logic.c

Chapter 5

Data Structure Documentation

5.1 app_snapshot_t Struct Reference

#include <app_runtime.h>

Data Fields

- int32_t myvar1
- bool connected
- uint32_t heartbeat

5.1.1 Detailed Description

Definition at line 11 of file app_runtime.h.

5.1.2 Field Documentation

5.1.2.1 connected

bool app_snapshot_t::connected

Definition at line 13 of file app_runtime.h.

5.1.2.2 heartbeat

uint32_t app_snapshot_t::heartbeat

Definition at line 14 of file app_runtime.h.

5.1.2.3 myvar1

int32_t app_snapshot_t::myvar1

Definition at line 12 of file app_runtime.h.

Referenced by ui_runtime_sync_timer_cb().

The documentation for this struct was generated from the following file:

- app_runtime.h

5.2 backend_handle_t Union Reference

#include <backends.h>

Data Fields

- display_backend_t * display
- indev_backend_t * indev

5.2.1 Detailed Description

Definition at line 51 of file backends.h.

5.2.2 Field Documentation

5.2.2.1 display

display_backend_t* backend_handle_t::display

Definition at line 52 of file backends.h.

Referenced by driver_backends_init_backend().

5.2.2.2 indev

indev_backend_t* backend_handle_t::indev

Definition at line 53 of file backends.h.

Referenced by driver_backends_init_backend().

The documentation for this union was generated from the following file:

- backends.h

5.3 backend_t Struct Reference

#include <backends.h>

Data Fields

- backend_handle_t * handle
- char * name
- backend_type_t type

5.3.1 Detailed Description

Definition at line 63 of file backends.h.

5.3.2 Field Documentation

5.3.2.1 handle

backend_handle_t* backend_t::handle

Definition at line 64 of file backends.h.

Referenced by driver_backends_init_backend(), and driver_backends_register().

5.3.2.2 name

char* backend_t::name

Definition at line 65 of file backends.h.

Referenced by driver_backends_init_backend(), driver_backends_is_supported(), and driver_backends_print_supporte

5.3.2.3 type

backend_type_t backend_t::type

Definition at line 66 of file backends.h.

Referenced by driver_backends_init_backend().

The documentation for this struct was generated from the following file:

- backends.h

5.4 display_backend_t Struct Reference

#include <backends.h>

Data Fields

- display_init_t init_display
- run_loop_t run_loop
- lv_display_t * display

5.4.1 Detailed Description

Definition at line 36 of file backends.h.

5.4.2 Field Documentation

5.4.2.1 display

lv_display_t* display_backend_t::display

Definition at line 39 of file backends.h.

Referenced by driver_backends_init_backend().

5.4.2.2 init_display

display_init_t display_backend_t::init_display

Definition at line 37 of file backends.h.

Referenced by driver_backends_init_backend().

5.4.2.3 run_loop

run_loop_t display_backend_t::run_loop

Definition at line 38 of file backends.h.

Referenced by driver_backends_run_loop().

The documentation for this struct was generated from the following file:

- backends.h

5.5 indev_backend_t Struct Reference

#include <backends.h>

Data Fields

- indev_init_t init_indev

5.5.1 Detailed Description

Definition at line 46 of file backends.h.

5.5.2 Field Documentation

5.5.2.1 init_indev

indev_init_t indev_backend_t::init_indev

Definition at line 47 of file backends.h.

Referenced by driver_backends_init_backend().

The documentation for this struct was generated from the following file:

- backends.h

5.6 simulator_settings_t Struct Reference

#include <simulator_settings.h>

Data Fields

- uint32_t window_width
- uint32_t window_height
- lv_display_rotation_t rotation
- bool maximize
- bool fullscreen

5.6.1 Detailed Description

Definition at line 35 of file simulator_settings.h.

5.6.2 Field Documentation

5.6.2.1 fullscreen

bool simulator_settings_t::fullscreen

Definition at line 40 of file simulator_settings.h.

5.6.2.2 maximize

bool simulator_settings_t::maximize

Definition at line 39 of file simulator_settings.h.

5.6.2.3 rotation

lv_display_rotation_t simulator_settings_t::rotation

Definition at line 38 of file simulator_settings.h.

5.6.2.4 window_height

uint32_t simulator_settings_t::window_height

Definition at line 37 of file simulator_settings.h.

5.6.2.5 window_width

uint32_t simulator_settings_t::window_width

Definition at line 36 of file simulator_settings.h.

The documentation for this struct was generated from the following file:

- simulator_settings.h

Chapter 6

File Documentation

6.1 app_runtime.cpp File Reference

#include "app_runtime.h"
#include <atomic>
#include <chrono>
#include <mutex>
#include <thread>

Functions

- int app_runtime_init (void)
- int app_runtime_start (void)
- void app_runtime_stop (void)
- void app_runtime_get_snapshot (app_snapshot_t *out_snapshot)

6.1.1 Function Documentation

6.1.1.1 app_runtime_get_snapshot()

```
void app_runtime_get_snapshot (
            app_snapshot_t * out_snapshot)
```

Definition at line 100 of file app_runtime.cpp.

Referenced by ui_runtime_sync_timer_cb().

6.1.1.2 app_runtime_init()

```
int app_runtime_init (
            void )
```

Definition at line 85 of file app_runtime.cpp.

Referenced by main().

6.1.1.3 app_runtime_start()

```
int app_runtime_start (
            void )
```

Definition at line 90 of file app_runtime.cpp.

Referenced by main().

6.1.1.4 app_runtime_stop()

```
void app_runtime_stop (
            void )
```

Definition at line 95 of file app_runtime.cpp.

Referenced by main().

6.2 app_runtime.cpp

Go to the documentation of this file.

```
00001 #include "app_runtime.h"
00002
00003 #include <atomic>
00004 #include <chrono>
00005 #include <mutex>
00006 #include <thread>
00007
00008 namespace {
00009
00010 class AppRuntime {
00011 public:
00012     int init()
00013     {
00014         std::lock_guard<std::mutex> lock(snapshot_mutex_);
00015         snapshot_ = {};
00016         snapshot_.connected = true;
00017         return 0;
00018     }
00019
00020     int start()
00021     {
00022         bool expected = false;
00023         if(!running_.compare_exchange_strong(expected, true)) {
00024             return 0;
00025         }
00026
00027         stop_requested_.store(false);
00028         worker_thread_ = std::thread(&AppRuntime::worker_loop, this);
00029         return 0;
00030     }
00031
00032     void stop()
00033     {
00034         if(!running_.load()) {
00035             return;
00036         }
00037
00038         stop_requested_.store(true);
00039         if(worker_thread_.joinable()) {
00040             worker_thread_.join();
00041         }
00042
00043         running_.store(false);
00044     }
00045
00046     void get_snapshot(app_snapshot_t * out_snapshot)
00047     {
00048         if(out_snapshot == nullptr) {
00049             return;
```

```
        }

        std::lock_guard<std::mutex> lock(snapshot_mutex_);
        *out_snapshot = snapshot_;
    }

private:
    void worker_loop()
    {
        while(!stop_requested_.load()) {
            {
                std::lock_guard<std::mutex> lock(snapshot_mutex_);
                snapshot_.myvar1 += 1;
                snapshot_.heartbeat += 1;
            }

            std::this_thread::sleep_for(std::chrono::milliseconds(500));
        }
    }

    std::atomic<bool> running_ { false };
    std::atomic<bool> stop_requested_ { false };
    std::mutex snapshot_mutex_;
    std::thread worker_thread_;
    app_snapshot_t snapshot_ {};
};

AppRuntime & runtime_instance()
{
    static AppRuntime runtime;
    return runtime;
}

} /* namespace */

extern "C" int app_runtime_init(void)
{
    return runtime_instance().init();
}

extern "C" int app_runtime_start(void)
{
    return runtime_instance().start();
}

extern "C" void app_runtime_stop(void)
{
    runtime_instance().stop();
}

extern "C" void app_runtime_get_snapshot(app_snapshot_t * out_snapshot)
{
    runtime_instance().get_snapshot(out_snapshot);
}
```

6.3 app_runtime.h File Reference

#include <stdbool.h>
#include <stdint.h>

Data Structures

- struct app_snapshot_t

Functions

- int app_runtime_init (void)
- int app_runtime_start (void)
- void app_runtime_stop (void)
- void app_runtime_get_snapshot (app_snapshot_t *out_snapshot)

6.3.1 Function Documentation

6.3.1.1 app_runtime_get_snapshot()

```
void app_runtime_get_snapshot (
            app_snapshot_t * out_snapshot)
```

Definition at line 100 of file app_runtime.cpp.

Referenced by ui_runtime_sync_timer_cb().

6.3.1.2 app_runtime_init()

```
int app_runtime_init (
            void )
```

Definition at line 85 of file app_runtime.cpp.

Referenced by main().

6.3.1.3 app_runtime_start()

```
int app_runtime_start (
            void )
```

Definition at line 90 of file app_runtime.cpp.

Referenced by main().

6.3.1.4 app_runtime_stop()

```
void app_runtime_stop (
            void )
```

Definition at line 95 of file app_runtime.cpp.

Referenced by main().

6.4 app_runtime.h

Go to the documentation of this file.

```
#ifndef APP_RUNTIME_H
#define APP_RUNTIME_H

#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

typedef struct {
    int32_t myvar1;
    bool connected;
    uint32_t heartbeat;
} app_snapshot_t;

int app_runtime_init(void);
int app_runtime_start(void);
void app_runtime_stop(void);
void app_runtime_get_snapshot(app_snapshot_t * out_snapshot);

#ifdef __cplusplus
}
#endif

#endif
```

6.5 backends.h File Reference

Interface for abstraction layer of a device backend.

Data Structures

- struct display_backend_t
- struct indev_backend_t
- union backend_handle_t
- struct backend_t

Typedefs

- typedef lv_display_t *(* display_init_t) (void)
- typedef void(* run_loop_t) (void)
- typedef lv_indev_t *(* indev_init_t) (lv_display_t *display)
- typedef int(* backend_init_t) (backend_t *)

Enumerations

- enum backend_type_t { BACKEND_DISPLAY , BACKEND_INDEV }

Functions

- int backend_init_fbdev (backend_t *backend)
- int backend_init_drm (backend_t *backend)
- int backend_init_sdl (backend_t *backend)
- int backend_init_glfw3 (backend_t *backend)
- int backend_init_wayland (backend_t *backend)
- int backend_init_x11 (backend_t *backend)
- int backend_init_evdev (backend_t *backend)

6.5.1 Detailed Description

Interface for abstraction layer of a device backend.

Copyright (c) 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file backends.h.

6.5.2 Typedef Documentation

6.5.2.1 backend_init_t

typedef int(* backend_init_t) (backend_t *)

Definition at line 70 of file backends.h.

6.5.2.2 display_init_t

typedef lv_display_t *(* display_init_t) (void)

Definition at line 30 of file backends.h.

6.5.2.3 indev_init_t

typedef lv_indev_t *(* indev_init_t) (lv_display_t *display)

Definition at line 43 of file backends.h.

6.5.2.4 run_loop_t

typedef void(* run_loop_t) (void)

Definition at line 33 of file backends.h.

6.5.3 Enumeration Type Documentation

6.5.3.1 backend_type_t

enum backend_type_t

Enumerator

BACKEND_DISPLAY	
BACKEND_INDEV	

Definition at line 57 of file backends.h.

6.5.4 Function Documentation

6.5.4.1 backend_init_drm()

```
int backend_init_drm (
            backend_t * backend)
```

6.5.4.2 backend_init_evdev()

```
int backend_init_evdev (
            backend_t * backend)
```

6.5.4.3 backend_init_fbdev()

```
int backend_init_fbdev (
            backend_t * backend)
```

6.5.4.4 backend_init_glfw3()

```
int backend_init_glfw3 (
            backend_t * backend)
```

6.5.4.5 backend_init_sdl()

```
int backend_init_sdl (
            backend_t * backend)
```

6.5.4.6 backend_init_wayland()

```
int backend_init_wayland (
            backend_t * backend)
```

6.5.4.7 backend_init_x11()

```
int backend_init_x11 (
            backend_t * backend)
```

6.6 backends.h

Go to the documentation of this file.

```
00011 #ifndef BACKENDS_H
00012 #define BACKENDS_H
00014 #ifdef __cplusplus
00015 extern "C" {
00016 #endif
00018 /********************
00019 *     INCLUDES
00020 ********************/
00022 /********************
00023 *     DEFINES
00024 ********************/
00026 /**********************
00027 *     TYPEDEFS
00028 **********************/
00029 /* Prototype of the display initialization functions */
00030 typedef lv_display_t * (*display_init_t)(void);
00032 /* Prototype of the run loop */
00033 typedef void (*run_loop_t)(void);
00035 /* Represents a display driver handle */
00036 typedef struct {
00037     display_init_t init_display; /* The display creation/initialization function */
00038     run_loop_t run_loop;         /* The run loop of the driver handle */
00039     lv_display_t * display;      /* The LVGL display that was created */
00040 } display_backend_t;
```

```
/* Prototype for the initialization of an indev driver backend */
typedef lv_indev_t * (*indev_init_t)(lv_display_t * display);

/* Represents an indev driver backend */
typedef struct {
    indev_init_t init_indev;
} indev_backend_t;

/* Regroup all different types of driver backend */
typedef union {
    display_backend_t * display;
    indev_backend_t * indev;
} backend_handle_t;

/* Define each type of driver backend */
typedef enum {
    BACKEND_DISPLAY,
    BACKEND_INDEV
} backend_type_t;

/* Driver backend descriptor */
typedef struct {
    backend_handle_t * handle;
    char * name;
    backend_type_t type;
} backend_t;

/* Prototype used to register a backend */
typedef int (*backend_init_t)(backend_t *);

/**********************
 * GLOBAL PROTOTYPES
 **********************/

/* Graphics backends */
int backend_init_fbdev(backend_t * backend);
int backend_init_drm(backend_t * backend);
int backend_init_sdl(backend_t * backend);
int backend_init_glfw3(backend_t * backend);
int backend_init_wayland(backend_t * backend);
int backend_init_x11(backend_t * backend);

/* Input device driver backends */
int backend_init_evdev(backend_t * backend);

/**********************
 *      MACROS
 **********************/

#ifdef __cplusplus
} /*extern "C"*/
#endif

#endif /*BACKEND_H*/
```

6.7 drm.c File Reference

The DRM/KMS backend.

#include <unistd.h>
#include <stdlib.h>
#include <stdbool.h>
#include "lvgl/lvgl.h"

6.7.1 Detailed Description

The DRM/KMS backend.

Based on the original file from the repository

- Move to a separate file 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file drm.c.

6.8 drm.c

Go to the documentation of this file.

```
00015
00016 /*********************
00017 *      INCLUDES
00018 *********************/
00019 #include <unistd.h>
00020 #include <stdlib.h>
00021 #include <stdbool.h>
00022
00023 #include "lvgl/lvgl.h"
00024 #if LV_USE_LINUX_DRM
00025 #include "../simulator_util.h"
00026 #include "../simulator_settings.h"
00027 #include "../backends.h"
00028
00029 /*********************
00030 *      DEFINES
00031 *********************/
00032
00033 /**********************
00034 *      TYPEDEFS
00035 **********************/
00036
00037 /**********************
00038 *  STATIC PROTOTYPES
00039 **********************/
00040 static void run_loop_drm(void);
00041 static lv_display_t * init_drm(void);
00042
00043
00044 /**********************
00045 *  STATIC VARIABLES
00046 **********************/
00047 static char * backend_name = "DRM";
00048
00049 /**********************
00050 *      MACROS
00051 **********************/
00052
00053 /**********************
00054 *   GLOBAL FUNCTIONS
00055 **********************/
00056
00063 int backend_init_drm(backend_t * backend)
00064 {
00065     LV_ASSERT_NULL(backend);
00066
00067     backend->handle->display = malloc(sizeof(display_backend_t));
00068     LV_ASSERT_NULL(backend->handle->display);
00069
00070     backend->handle->display->init_display = init_drm;
00071     backend->handle->display->run_loop = run_loop_drm;
00072     backend->name = backend_name;
00073     backend->type = BACKEND_DISPLAY;
00074
00075     return 0;
00076 }
00077
00078 /**********************
00079 *   STATIC FUNCTIONS
00080 **********************/
00081
00087 static lv_display_t * init_drm(void)
00088 {
00089     const char * device = getenv_default("LV_LINUX_DRM_CARD", lv_linux_drm_find_device_path());
00090     lv_display_t * disp = lv_linux_drm_create();
00091
00092     if(disp == NULL) {
00093         return NULL;
```

```
    }

    lv_linux_drm_set_file(disp, device, -1);

    return disp;
}


static void run_loop_drm(void)
{
    uint32_t idle_time;

    /* Handle LVGL tasks */
    while(true) {
        /* Returns the time to the next timer execution */
        idle_time = lv_timer_handler();
        usleep(idle_time * 1000);
    }
}

#endif /*#if LV_USE_LINUX_DRM*/
```

6.9 fbdev.c File Reference

Legacy framebuffer device.

#include <unistd.h>
#include <stdlib.h>
#include <stdbool.h>
#include "lvgl/lvgl.h"

6.9.1 Detailed Description

Legacy framebuffer device.

Based on the original file from the repository

Move to a separate file 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file fbdev.c.

6.10 fbdev.c

Go to the documentation of this file.

```
/*********************
 *      INCLUDES
 *********************/
#include <unistd.h>
#include <stdlib.h>
#include <stdbool.h>

#include "lvgl/lvgl.h"
#if LV_USE_LINUX_FBDEV
#include "../simulator_util.h"
#include "../backends.h"

/*********************
 *      DEFINES
 *********************/
```

```c

/**********************
 *      TYPEDEFS
 **********************/

/**********************
 *  STATIC PROTOTYPES
 **********************/

static lv_display_t * init_fbdev(void);
static void run_loop_fbdev(void);

/**********************
 *  STATIC VARIABLES
 **********************/

static char * backend_name = "FBDEV";

/**********************
 *      MACROS
 **********************/

/**********************
 *   GLOBAL FUNCTIONS
 **********************/

int backend_init_fbdev(backend_t * backend)
{
    LV_ASSERT_NULL(backend);

    backend->handle->display = malloc(sizeof(display_backend_t));
    LV_ASSERT_NULL(backend->handle->display);

    backend->handle->display->init_display = init_fbdev;
    backend->handle->display->run_loop = run_loop_fbdev;
    backend->name = backend_name;
    backend->type = BACKEND_DISPLAY;

    return 0;
}

/**********************
 *   STATIC FUNCTIONS
 **********************/

static lv_display_t * init_fbdev(void)
{
    const char * device = getenv_default("LV_LINUX_FBDEV_DEVICE", "/dev/fb0");
    lv_display_t * disp = lv_linux_fbdev_create();

    if(disp == NULL) {
        return NULL;
    }

    lv_linux_fbdev_set_file(disp, device);

    return disp;
}

static void run_loop_fbdev(void)
{
    uint32_t idle_time;

    /* Handle LVGL tasks */
    while(true) {

        /* Returns the time to the next timer execution */
        idle_time = lv_timer_handler();
        usleep(idle_time * 1000);
    }
}

#endif /*LV_USE_LINUX_FBDEV*/
```

6.11 glfw3.c File Reference

The GLFW3 backend.

#include <unistd.h>
#include <stdlib.h>

#include <stdbool.h>
#include "lvgl/lvgl.h"

6.11.1 Detailed Description

The GLFW3 backend.

Based on the original file from the repository

Move to a separate file 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file glfw3.c.

6.12 glfw3.c

Go to the documentation of this file.

```
00001
00014
00015 /*********************
00016  *      INCLUDES
00017  *********************/
00018
00019 #include <unistd.h>
00020 #include <stdlib.h>
00021 #include <stdbool.h>
00022
00023 #include "lvgl/lvgl.h"
00024 #if LV_USE_GLFW
00025 #include "../simulator_util.h"
00026 #include "../simulator_settings.h"
00027 #include "../backends.h"
00028
00029 /*********************
00030  *      DEFINES
00031  *********************/
00032
00033 /**********************
00034  *      TYPEDEFS
00035  **********************/
00036
00037 /**********************
00038  *  STATIC PROTOTYPES
00039  **********************/
00040 static void run_loop_glfw3(void);
00041 static lv_display_t * init_glfw3(void);
00042
00043 /*********************
00044  *  STATIC VARIABLES
00045  *********************/
00046 static char * backend_name = "GLFW";
00047
00048 /*********************
00049  *  EXTERNAL VARIABLES
00050  **********************/
00051 extern simulator_settings_t settings;
00052
00053 /*********************
00054  *      MACROS
00055  *********************/
00056
00057 /**********************
00058  *   GLOBAL FUNCTIONS
00059  **********************/
00060
00067 int backend_init_glfw3(backend_t * backend)
00068 {
00069
00070     LV_ASSERT_NULL(backend);
00071     backend->handle->display = malloc(sizeof(display_backend_t));
```

```

    LV_ASSERT_NULL(backend->handle->display);

    backend->handle->display->init_display = init_glfw3;
    backend->handle->display->run_loop = run_loop_glfw3;
    backend->name = backend_name;
    backend->type = BACKEND_DISPLAY;

    return 0;

}

/**********************
 *   STATIC FUNCTIONS
 **********************/

static lv_display_t * init_glfw3(void)
{
    lv_glfw_texture_t * window_texture;
    lv_indev_t * mouse;
    lv_display_t * disp_texture;
    uint32_t disp_texture_id;
    lv_obj_t * cursor_obj;

    /* create a window and initialize OpenGL */
    lv_glfw_window_t * window = lv_glfw_window_create(
                                settings.window_width, settings.window_height, true);

    /* create a display that flushes to a texture */
    disp_texture = lv_opengles_texture_create(
                        settings.window_width, settings.window_height);
    lv_display_set_default(disp_texture);

    /* add the texture to the window */
    disp_texture_id = lv_opengles_texture_get_texture_id(disp_texture);
    window_texture = lv_glfw_window_add_texture(window, disp_texture_id,
                                        settings.window_width, settings.window_height);

    /* get the mouse indev of the window texture */
    mouse = lv_glfw_texture_get_mouse_indev(window_texture);

    /* add a cursor to the mouse indev */
    LV_IMAGE_DECLARE(mouse_cursor_icon);
    cursor_obj = lv_image_create(lv_screen_active());
    lv_image_set_src(cursor_obj, &mouse_cursor_icon);
    lv_indev_set_cursor(mouse, cursor_obj);

    return disp_texture;
}

void run_loop_glfw3(void)
{
    uint32_t idle_time;

    /* Handle LVGL tasks */
    while(true) {

        /* Returns the time to the next timer execution */
        idle_time = lv_timer_handler();
        usleep(idle_time * 1000);
    }
}

#endif /*#if LV_USE_GLFW*/
```

6.13 sdl.c File Reference

The backend for the SDL simulator.

#include <unistd.h>
#include <stdlib.h>
#include <stdbool.h>
#include "lvgl/lvgl.h"

6.13.1 Detailed Description

The backend for the SDL simulator.

Based on the original file from the repository

- Move to a separate file 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file sdl.c.

6.14 sdl.c

Go to the documentation of this file.

```
/*********************
 *      INCLUDES
 *********************/
#include <unistd.h>
#include <stdlib.h>
#include <stdbool.h>

#include "lvgl/lvgl.h"
#if LV_USE_SDL
#include "../simulator_util.h"
#include "../simulator_settings.h"
#include "../backends.h"

/*********************
 *      DEFINES
 *********************/

/*********************
 *      TYPEDEFS
 *********************/

/**********************
 *  EXTERNAL VARIABLES
 **********************/
extern simulator_settings_t settings;

/**********************
 *  STATIC PROTOTYPES
 **********************/
static void run_loop_sdl(void);
static lv_display_t * init_sdl(void);

/**********************
 *  STATIC VARIABLES
 **********************/

static char * backend_name = "SDL";

/*********************
 *      MACROS
 *********************/

/**********************
 *   GLOBAL FUNCTIONS
 **********************/

int backend_init_sdl(backend_t * backend)
{
    LV_ASSERT_NULL(backend);

    backend->handle->display = malloc(sizeof(display_backend_t));
    LV_ASSERT_NULL(backend->handle->display);

    backend->handle->display->init_display = init_sdl;
    backend->handle->display->run_loop = run_loop_sdl;
    backend->name = backend_name;
```

```
    backend->type = BACKEND_DISPLAY;

    return 0;
}

/**********************
 *   STATIC FUNCTIONS
 **********************/

static lv_display_t * init_sdl(void)
{

    lv_display_t * disp = lv_sdl_window_create(settings.window_width, settings.window_height);

    if(disp == NULL) {
        return NULL;
    }


    lv_indev_t * mouse = lv_sdl_mouse_create();
    lv_indev_set_group(mouse, lv_group_get_default());
    lv_indev_set_display(mouse, disp);

    LV_IMAGE_DECLARE(mouse_cursor_icon);
    lv_obj_t * cursor_obj;
    cursor_obj = lv_image_create(lv_screen_active());
    lv_image_set_src(cursor_obj, &mouse_cursor_icon);
    lv_indev_set_cursor(mouse, cursor_obj);

    lv_indev_t * mousewheel = lv_sdl_mousewheel_create();
    lv_indev_set_display(mousewheel, disp);
    lv_indev_set_group(mousewheel, lv_group_get_default());

    lv_indev_t * kb = lv_sdl_keyboard_create();
    lv_indev_set_display(kb, disp);
    lv_indev_set_group(kb, lv_group_get_default());



    return disp;
}

static void run_loop_sdl(void)
{
    uint32_t idle_time;

    /* Handle LVGL tasks */
    while(true) {
        /* Returns the time to the next timer execution */
        idle_time = lv_timer_handler();
        usleep(idle_time * 1000);
    }
}
#endif /*#if LV_USE_SDL*/
```

6.15 wayland.c File Reference

The wayland backend.

#include <unistd.h>
#include <stdbool.h>
#include <stdlib.h>
#include "lvgl/lvgl.h"

6.15.1 Detailed Description

The wayland backend.

Based on the original file from the repository

- Move to a separate file 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file wayland.c.

6.16 wayland.c

Go to the documentation of this file.

```
/*********************
 *      INCLUDES
 *********************/
#include <unistd.h>
#include <stdbool.h>
#include <stdlib.h>

#include "lvgl/lvgl.h"
#if LV_USE_WAYLAND
#include "../simulator_util.h"
#include "../simulator_settings.h"
#include "../backends.h"

/*********************
 *      DEFINES
 *********************/

/**********************
 *      TYPEDEFS
 **********************/

/**********************
 *  STATIC PROTOTYPES
 **********************/
static lv_display_t * init_wayland(void);
static void run_loop_wayland(void);

/**********************
 *  STATIC VARIABLES
 **********************/
static char * backend_name = "WAYLAND";

/**********************
 *  EXTERNAL VARIABLES
 **********************/
extern simulator_settings_t settings;

/**********************
 *      MACROS
 **********************/

/**********************
 *   GLOBAL FUNCTIONS
 **********************/

int backend_init_wayland(backend_t * backend)
{
    LV_ASSERT_NULL(backend);
    backend->handle->display = malloc(sizeof(display_backend_t));
    LV_ASSERT_NULL(backend->handle->display);

    backend->handle->display->init_display = init_wayland;
    backend->handle->display->run_loop = run_loop_wayland;
    backend->name = backend_name;
    backend->type = BACKEND_DISPLAY;

    return 0;
}

/**********************
 *   STATIC FUNCTIONS
 **********************/

static lv_display_t * init_wayland(void)
{
    lv_display_t * disp;
    lv_group_t * g;

    disp = lv_wayland_window_create(settings.window_width, settings.window_height,
                                    "LVGL Simulator", NULL);

    if(disp == NULL) {
        die("Failed to initialize Wayland backend\n");
    }

    if(settings.fullscreen) {
        lv_wayland_window_set_fullscreen(disp, true);
    }
    else if(settings.maximize) {
        lv_wayland_window_set_maximized(disp, true);
```

```
    }

    g = lv_group_create();
    lv_group_set_default(g);
    lv_indev_set_group(lv_wayland_get_keyboard(disp), g);
    lv_indev_set_group(lv_wayland_get_pointeraxis(disp), g);

    return disp;

}

static void run_loop_wayland(void)
{

    uint32_t idle_time;

    /* Handle LVGL tasks */
    while(true) {

        idle_time = lv_wayland_timer_handler();

        if(idle_time != 0) {
            usleep(idle_time * 1000);
        }
        /* Run until the last window closes */
        if(!lv_wayland_window_is_open(NULL)) {
            break;
        }
    }
}

#endif /*#if LV_USE_WAYLAND*/
```

6.17 x11.c File Reference

The backend for the X11 windowing system.

#include <unistd.h>
#include <stdlib.h>
#include <stdbool.h>
#include "lvgl/lvgl.h"

6.17.1 Detailed Description

The backend for the X11 windowing system.

Based on the original file from the repository.

Move to a separate file 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file x11.c.

6.18 x11.c

Go to the documentation of this file.

```
/*********************
 *      INCLUDES
 *********************/

#include <unistd.h>
#include <stdlib.h>
#include <stdbool.h>

#include "lvgl/lvgl.h"
#if LV_USE_X11
#include "../simulator_util.h"
#include "../simulator_settings.h"
#include "../backends.h"

/*********************
 *      DEFINES
 *********************/

/**********************
 *      TYPEDEFS
 **********************/

/**********************
 *  EXTERNAL VARIABLES
 **********************/
extern simulator_settings_t settings;

/**********************
 *  STATIC PROTOTYPES
 **********************/
static lv_display_t * init_x11(void);
static void run_loop_x11(void);

/**********************
 *  STATIC VARIABLES
 **********************/
static char * backend_name = "X11";

/**********************
 *      MACROS
 **********************/

/**********************
 *   GLOBAL FUNCTIONS
 **********************/

int backend_init_x11(backend_t * backend)
{
    LV_ASSERT_NULL(backend);
    backend->handle->display = malloc(sizeof(display_backend_t));
    LV_ASSERT_NULL(backend->handle->display);

    backend->name = backend_name;
    backend->handle->display->init_display = init_x11;
    backend->handle->display->run_loop = run_loop_x11;
    backend->type = BACKEND_DISPLAY;

    return 0;
}

/**********************
 *   STATIC FUNCTIONS
 **********************/

static lv_display_t * init_x11(void)
{
    lv_display_t * disp;
    LV_IMG_DECLARE(mouse_cursor_icon);

    disp = lv_x11_window_create("LVGL simulator",
                                settings.window_width, settings.window_height);

    disp = lv_display_get_default();

    if(disp == NULL) {
        return NULL;
    }

    lv_x11_inputs_create(disp, &mouse_cursor_icon);

```

```
    return disp;
}

void run_loop_x11(void)
{
    uint32_t idle_time;

    /* Handle LVGL tasks */
    while(true) {
        /* Returns the time to the next timer execution */
        idle_time = lv_timer_handler();
        usleep(idle_time * 1000);
    }
}

#endif /*#if LV_USE_X11*/
```

6.19 driver_backends.c File Reference

Copyright (c) 2025 EDGEMTech Ltd.

#include <unistd.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <ctype.h>
#include "lvgl/lvgl.h"
#include "simulator_util.h"
#include "simulator_settings.h"
#include "driver_backends.h"
#include "backends.h"

Functions

- void driver_backends_register (void)
- int driver_backends_init_backend (char *backend_name)

  Initialize the specified backend in case of a display driver backend.
- int driver_backends_print_supported (void)

  Print supported backends Prints a list of supported backends.
- int driver_backends_is_supported (char *backend_name)

  Checks if a backend exists and is supported.
- void driver_backends_run_loop (void)

  Enter the run loop enter the run loop of the selected backend.

Variables

- backend_init_t available_backends []
- static backend_t * backends [sizeof(available_backends)/sizeof(available_backends[0])]
- static backend_t * sel_display_backend = NULL
- simulator_settings_t settings

6.19.1 Detailed Description

Copyright (c) 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file driver_backends.c.

6.19.2 Function Documentation

6.19.2.1 driver_backends_init_backend()

```
int driver_backends_init_backend (
            char * backend_name)
```

Initialize the specified backend in case of a display driver backend.

- create the lv_display, in case of a indev driver backend create an input device

Parameters

backend_name	the name of the backend to initialize FBDEV,DRM etc

Returns

0 on success, -1 on error

Definition at line 141 of file driver_backends.c.

References BACKEND_DISPLAY, BACKEND_INDEV, backends, backend_handle_t::display, display_backend_t::display, backend_t::handle, backend_handle_t::indev, display_backend_t::init_display, indev_backend_t::init_indev, backend_t::name, sel_display_backend, and backend_t::type.

Referenced by main().

6.19.2.2 driver_backends_is_supported()

```
int driver_backends_is_supported (
            char * backend_name)
```

Checks if a backend exists and is supported.

Parameters

backend_name	the backend name to check

Returns

1 is supported, 0 not supported or invalid name

Definition at line 247 of file driver_backends.c.

References backends, and backend_t::name.

Referenced by configure_simulator().

6.19.2.3 driver_backends_print_supported()

```
int driver_backends_print_supported (
            void )
```

Print supported backends Prints a list of supported backends.

Returns

-1 if an error occurred, 0 on success

Definition at line 222 of file driver_backends.c.

References backends, and backend_t::name.

Referenced by configure_simulator().

6.19.2.4 driver_backends_register()

```
void driver_backends_register (
            void )
```

Definition at line 113 of file driver_backends.c.

References available_backends, backends, and backend_t::handle.

Referenced by configure_simulator(), and main().

6.19.2.5 driver_backends_run_loop()

```
void driver_backends_run_loop (
            void )
```

Enter the run loop enter the run loop of the selected backend.

Definition at line 268 of file driver_backends.c.

References display_backend_t::run_loop, and sel_display_backend.

Referenced by main().

6.19.3 Variable Documentation

6.19.3.1 available_backends

backend_init_t available_backends[]

Definition at line 61 of file driver_backends.c.

Referenced by driver_backends_register().

6.19.3.2 backends

backend_t* backends[sizeof(available_backends)/sizeof(available_backends[0])] [static]

Definition at line 94 of file driver_backends.c.

Referenced by driver_backends_init_backend(), driver_backends_is_supported(), driver_backends_print_supported(), and driver_backends_register().

6.19.3.3 sel_display_backend

backend_t* sel_display_backend = NULL [static]

Definition at line 97 of file driver_backends.c.

Referenced by driver_backends_init_backend(), and driver_backends_run_loop().

6.19.3.4 settings

simulator_settings_t settings

Definition at line 103 of file driver_backends.c.

Referenced by configure_simulator(), and main().

6.20 driver_backends.c

Go to the documentation of this file.

```
/*********************
 *      INCLUDES
 *********************/

#include <unistd.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <ctype.h>

#include "lvgl/lvgl.h"

#include "simulator_util.h"
#include "simulator_settings.h"
#include "driver_backends.h"

#include "backends.h"

/*********************
 *      DEFINES
 *********************/

/* Catch configuration errors at compile time - checks if no backend was selected */
#if LV_USE_SDL == 0 && \
    LV_USE_WAYLAND == 0 && \
    LV_USE_LINUX_DRM == 0 && \
    LV_USE_GLFW == 0 && \
    LV_USE_X11 == 0 && \
    LV_USE_LINUX_FBDEV == 0

    #error Unsupported configuration - Please select at least one graphics backend in lv_conf.h
#endif

/**********************
 *      TYPEDEFS
 **********************/

/**********************
 *  STATIC PROTOTYPES
 **********************/

/**********************
 *  STATIC VARIABLES
 **********************/

/* The default backend is the one that will end up at index 0
 * To add support for a new driver backend add the declaration
 * and append an entry to the available_backends array
 */
backend_init_t available_backends[] = {

#if LV_USE_LINUX_FBDEV
    backend_init_fbdev,
#endif

#if LV_USE_LINUX_DRM
    backend_init_drm,
#endif

#if LV_USE_SDL
    backend_init_sdl,
#endif

#if LV_USE_WAYLAND
    backend_init_wayland,
#endif

#if LV_USE_X11
    backend_init_x11,
#endif

#if LV_USE_GLFW
    backend_init_glfw3,
#endif

#if LV_USE_EVDEV
    backend_init_evdev,
```

```
#endif
    NULL    /* Sentinel */
};

/* Contains the backend descriptors */
static backend_t * backends[sizeof(available_backends) / sizeof(available_backends[0])];

/* Set once the user selects a backend - or it is set to the default backend */
static backend_t * sel_display_backend = NULL;

/**********************
 *  GLOBAL VARIABLES
 **********************/
/* Contains global simulator settings common to each backend */
simulator_settings_t settings;

/**********************
 *      MACROS
 **********************/

/**********************
 *   GLOBAL FUNCTIONS
 **********************/

void driver_backends_register(void)
{
    int i;
    backend_init_t init_backend;
    backend_t * b;

    i = 0;
    if(backends[i] != NULL) {
        /* backends are already registered - leave */
        return;
    }

    while((init_backend = available_backends[i]) != NULL) {

        b = malloc(sizeof(backend_t));
        LV_ASSERT_NULL(b);

        b->handle = malloc(sizeof(backend_handle_t));

        init_backend = available_backends[i];
        LV_ASSERT_NULL(init_backend);

        init_backend(b);
        backends[i] = b;
        i++;
    }
}

int driver_backends_init_backend(char * backend_name)
{
    backend_t * b;
    int i;
    display_backend_t * dispb;
    indev_backend_t * indevb;

    if(backends[0] == NULL) {
        LV_LOG_ERROR("Please call driver_backends_register first");
        return -1;
    }

    if(backend_name == NULL) {

        /*
         * Set default display backend - which is the first defined
         * item in available_backends array
         */
        LV_ASSERT_NULL(backends[0]);
        b = backends[0];

        if(b->type != BACKEND_DISPLAY) {
            LV_LOG_ERROR("The default backend: %s is not a display driver backend", b->name);
            return -1;
        }

        backend_name = backends[0]->name;
    }

    i = 0;
    while((b = backends[i]) != NULL) {

        /* Check if such a backend exists */
        if(strcmp(b->name, backend_name) == 0) {

```

```
            if(b->type == BACKEND_DISPLAY) {
                /* Initialize the display */

                dispb = b->handle->display;
                LV_ASSERT_NULL(dispb->init_display);
                dispb->display = dispb->init_display();

                if(dispb->display == NULL) {
                    LV_LOG_ERROR("Failed to init display with %s backend", b->name);
                    return -1;
                }

                sel_display_backend = b;
                LV_LOG_INFO("Initialized %s display backend", b->name);
                break;

            }
            else if(b->type == BACKEND_INDEV) {
                /* Initialize input device */

                indevb = b->handle->indev;
                LV_ASSERT_NULL(indevb->init_indev);

                /* The display driver backend - has to be initialized first */
                if(sel_display_backend == NULL) {
                    LV_LOG_ERROR(
                        "Failed to init indev backend: %s - display needs to be initialized",
                        b->name);
                    return -1;
                }

                LV_LOG_INFO("Initialized %s indev backend", b->name);

                dispb = sel_display_backend->handle->display;

                LV_ASSERT_NULL(dispb->display);
                indevb->init_indev(dispb->display);
                break;
            }
        }
        i++;
    }

    return 0;
}

int driver_backends_print_supported(void)
{
    int i;
    backend_t * b;

    i = 0;
    if(backends[i] == NULL) {
        LV_LOG_ERROR("Please call driver_backends_register first");
        return -1;
    }

    b = backends[i];

    fprintf(stdout, "Default backend: %s\n", b->name);
    fprintf(stdout, "Supported backends: ");

    while((b = backends[i++]) != NULL) {
        fprintf(stdout, "%s ", b->name);
    }

    fprintf(stdout, "\n");
    return 0;

}

int driver_backends_is_supported(char * backend_name)
{
    char c;
    backend_t * b;
    char * name = backend_name;
    int i = 0;

    while((c = *backend_name) != '\0') {
        *backend_name = toupper(c);
        backend_name++;
    }

    while((b = backends[i++]) != NULL) {
        if(strcmp(b->name, name) == 0) {
            return 1;
        }
```

```
    }

    return 0;
}

void driver_backends_run_loop(void)
{
    display_backend_t * dispb;

    if(sel_display_backend != NULL && sel_display_backend->handle->display != NULL) {

        dispb = sel_display_backend->handle->display;
        dispb->run_loop();

    }
    else {
        LV_LOG_ERROR("No backend has been selected - initialize the backend first");
    }
}

/**********************
*   STATIC FUNCTIONS
**********************/

```

6.21 driver_backends.h File Reference

provides an abstraction to support multiple graphical driver backends at the same time without recompiling everything each time

Functions

- void driver_backends_register (void)
- int driver_backends_init_backend (char *backend_name)

  Initialize the specified backend in case of a display driver backend.
- int driver_backends_is_supported (char *backend_name)

  Checks if a backend exists and is supported.
- int driver_backends_print_supported (void)

  Print supported backends Prints a list of supported backends.
- void driver_backends_run_loop (void)

  Enter the run loop enter the run loop of the selected backend.

6.21.1 Detailed Description

provides an abstraction to support multiple graphical driver backends at the same time without recompiling everything each time

E.g: this means LVGL can be compiled with both SDL or X11

- see backend.h for the details on the interface.
- see the files in display_backends directory for examples on how to use each driver

Copyright (c) 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file driver_backends.h.

6.21.2 Function Documentation

6.21.2.1 driver_backends_init_backend()

```
int driver_backends_init_backend (
            char * backend_name)
```

Initialize the specified backend in case of a display driver backend.

- create the lv_display, in case of a indev driver backend create an input device

Parameters

backend_name	the name of the backend to initialize FBDEV,DRM etc

Returns

0 on success, -1 on error

Definition at line 141 of file driver_backends.c.

References BACKEND_DISPLAY, BACKEND_INDEV, backends, backend_handle_t::display, display_backend_t::display, backend_t::handle, backend_handle_t::indev, display_backend_t::init_display, indev_backend_t::init_indev, backend_t::name, sel_display_backend, and backend_t::type.

Referenced by main().

6.21.2.2 driver_backends_is_supported()

```
int driver_backends_is_supported (
            char * backend_name)
```

Checks if a backend exists and is supported.

Parameters

backend_name	the backend name to check

Returns

1 is supported, 0 not supported or invalid name

Definition at line 247 of file driver_backends.c.

References backends, and backend_t::name.

Referenced by configure_simulator().

6.21.2.3 driver_backends_print_supported()

```
int driver_backends_print_supported (
            void )
```

Print supported backends Prints a list of supported backends.

Returns

-1 if an error occurred, 0 on success

Definition at line 222 of file driver_backends.c.

References backends, and backend_t::name.

Referenced by configure_simulator().

6.21.2.4 driver_backends_register()

```
void driver_backends_register (
            void )
```

Definition at line 113 of file driver_backends.c.

References available_backends, backends, and backend_t::handle.

Referenced by configure_simulator(), and main().

6.21.2.5 driver_backends_run_loop()

```
void driver_backends_run_loop (
            void )
```

Enter the run loop enter the run loop of the selected backend.

Definition at line 268 of file driver_backends.c.

References display_backend_t::run_loop, and sel_display_backend.

Referenced by main().

6.22 driver_backends.h

Go to the documentation of this file.

```
#ifndef DRIVER_BACKENDS_H
#define DRIVER_BACKENDS_H

#ifdef __cplusplus
extern "C" {
#endif

/*********************
 *      INCLUDES
 *********************/

/*********************
 *      DEFINES
 *********************/

/**********************
 *      TYPEDEFS
 **********************/

/**********************
 * GLOBAL PROTOTYPES
 **********************/

/*
 * Register all available backends
 * This function must be called first before any other
 * function
 */
void driver_backends_register(void);

int driver_backends_init_backend(char * backend_name);

int driver_backends_is_supported(char * backend_name);

int driver_backends_print_supported(void);

void driver_backends_run_loop(void);

/**********************
 *      MACROS
 **********************/

#ifdef __cplusplus
} /*extern "C"*/
#endif

#endif /*DRIVER_BACKENDS_H*/
```

6.23 evdev.c File Reference

The lib evdev driver.

#include <stdbool.h>
#include <stdlib.h>
#include <unistd.h>
#include "lvgl/lvgl.h"

6.23.1 Detailed Description

The lib evdev driver.

Based on the original file from the repository

- Move the driver to a separate file to avoid excessive conditional compilation Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Copyright (c) 2025 EDGEMTech Ltd.

Definition in file evdev.c.

6.24 evdev.c

Go to the documentation of this file.

```

/*********************
 *      INCLUDES
 *********************/
#include <stdbool.h>
#include <stdlib.h>
#include <unistd.h>

#include "lvgl/lvgl.h"
#if LV_USE_EVDEV
#include "lvgl/src/core/lv_global.h"
#include "../backends.h"

/*********************
 *      DEFINES
 *********************/

/**********************
 *      TYPEDEFS
 **********************/

/**********************
 *  STATIC PROTOTYPES
 **********************/

static void indev_deleted_cb(lv_event_t * e);
static void discovery_cb(lv_indev_t * indev, lv_evdev_type_t type, void * user_data);
static void set_mouse_cursor_icon(lv_indev_t * indev, lv_display_t * display);
static lv_indev_t * init_pointer_evdev(lv_display_t * display);

/**********************
 *  STATIC VARIABLES
 **********************/

static char * backend_name = "EVDEV";

/**********************
 *      MACROS
 **********************/

/**********************
 *   GLOBAL FUNCTIONS
 **********************/

/*
 * Initialize the evdev driver
 *
 * @param backend the backend descriptor
 */
int backend_init_evdev(backend_t * backend)
{
    LV_ASSERT_NULL(backend);
    backend->handle->indev = malloc(sizeof(indev_backend_t));
    LV_ASSERT_NULL(backend->handle->indev);

    backend->handle->indev->init_indev = init_pointer_evdev;

    backend->name = backend_name;
    backend->type = BACKEND_INDEV;
    return 0;
}


/**********************
 *   STATIC FUNCTIONS
 **********************/
```

```

/*
 * Remove cursor icon
 *
 * @description When the indev is deleted remove the mouse cursor icon
 * @note called by the LVGL evdev driver
 * @param e the deletion event
 */
static void indev_deleted_cb(lv_event_t * e)
{
    if(LV_GLOBAL_DEFAULT()->deinit_in_progress) return;
    lv_obj_t * cursor_obj = lv_event_get_user_data(e);
    lv_obj_delete(cursor_obj);
}


/*
 * Set cursor icon
 *
 * @description Once the input device is discovered set the mouse cursor icon
 * @note called by the LVGL evdev driver
 * @param indev the input device
 * @param type the type of the input device
 * @param user_data the user data
 */
static void discovery_cb(lv_indev_t * indev, lv_evdev_type_t type, void * user_data)
{
    LV_LOG_USER("new '%s' device discovered", type == LV_EVDEV_TYPE_REL ? "REL" :
                type == LV_EVDEV_TYPE_ABS ? "ABS" :
                type == LV_EVDEV_TYPE_KEY ? "KEY" :
                "unknown");

    lv_display_t * disp = user_data;
    lv_indev_set_display(indev, disp);

    if(type == LV_EVDEV_TYPE_REL) {
        set_mouse_cursor_icon(indev, disp);
    }
}

/*
 * Set cursor icon
 *
 * @description Enables a pointer (touchscreen/mouse) input device
 * @param display the display on which to create
 * @param indev the input device to set the cursor on
 */
static void set_mouse_cursor_icon(lv_indev_t * indev, lv_display_t * display)
{
    /* Set the cursor icon */
    LV_IMAGE_DECLARE(mouse_cursor_icon);
    lv_obj_t * cursor_obj = lv_image_create(lv_display_get_screen_active(display));
    lv_image_set_src(cursor_obj, &mouse_cursor_icon);
    lv_indev_set_cursor(indev, cursor_obj);

    /* delete the mouse cursor icon if the device is removed */
    lv_indev_add_event_cb(indev, indev_deleted_cb, LV_EVENT_DELETE, cursor_obj);

}

/*
 * Initialize a mouse pointer device
 *
 * Enables a pointer (touchscreen/mouse) input device
 * Use 'evtest' to find the correct input device. /dev/input/by-id/ is recommended if possible
 * Use /dev/input/by-id/my-mouse-or-touchscreen or /dev/input/eventX
 *
 * If LV_LINUX_EVDEV_POINTER_DEVICE is not set, automatic evdev discovery will start
 *
 * @param display the LVGL display
 *
 * @return input device
 */
static lv_indev_t * init_pointer_evdev(lv_display_t * display)
{
    const char * input_device = getenv("LV_LINUX_EVDEV_POINTER_DEVICE");

    if(input_device == NULL) {
        LV_LOG_USER("Using evdev automatic discovery.");
        lv_evdev_discovery_start(discovery_cb, display);
        return NULL;
    }

    lv_indev_t * indev = lv_evdev_create(LV_INDEV_TYPE_POINTER, input_device);

    if(indev == NULL) {
        return NULL;
```

```
    }

    lv_indev_set_display(indev, display);

    set_mouse_cursor_icon(indev, display);
    return indev;
}
#endif /*#if LV_USE_EVDEV*/
```

6.25 mouse_cursor_icon.c File Reference

The C array containing the mouse cursor icon.

#include "lvgl/lvgl.h"

Variables

- const uint8_t mouse_cursor_icon_map []
- lv_image_dsc_t mouse_cursor_icon

6.25.1 Detailed Description

The C array containing the mouse cursor icon.

Based on the original file from the repository Copyright (c) 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file mouse_cursor_icon.c.

6.25.2 Variable Documentation

6.25.2.1 mouse_cursor_icon

lv_image_dsc_t mouse_cursor_icon

Initial value:

```
                                = {
    .header.magic = LV_IMAGE_HEADER_MAGIC,
    .header.w = 14,
    .header.h = 20,
    .header.stride = 56,
    .data_size = 280 * 4,
    .header.cf = LV_COLOR_FORMAT_ARGB8888,
    .data = mouse_cursor_icon_map,
}
```

Definition at line 38 of file mouse_cursor_icon.c.

6.25.2.2 mouse_cursor_icon_map

const uint8_t mouse_cursor_icon_map[]

Initial value:

```
                                = {
    0x19, 0x19, 0x19, 0xb8, 0x1e, 0x1e, 0x1e, 0xc8, 0x00, 0x00, 0x00, 0x13, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x48, 0x48, 0x48, 0xcc, 0xb2, 0xb2, 0xb2, 0xff, 0x3a, 0x3a, 0x3a, 0xcc, 0x00, 0x00, 0x00, 0x24, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x0a, 0x0a, 0x0a, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3b, 0x3b, 0x3b, 0xc8, 0xf6, 0xf6, 0xf6, 0xff, 0xdc, 0xdc, 0xdc, 0xff, 0x43, 0x43, 0x43, 0xe0, 0x00, 0x00, 0x00, 0x3b, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0a, 0x0a, 0x0a, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3b, 0x3b, 0x3b, 0xcb, 0xe6, 0xe6, 0xe6, 0xff, 0xff, 0xff, 0xff, 0xfc, 0xe5, 0xe5, 0xe5, 0xff, 0x59, 0x59, 0x59, 0xf3, 0x00, 0x00,
        0x00, 0x4f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3c, 0x3c, 0x3c, 0xcb, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf5, 0xf5, 0xf5, 0xff, 0x72, 0x72, 0x72,
        0xff, 0x00, 0x00, 0x00, 0x73, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3d, 0x3d, 0x3d, 0xcb, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
        0xff, 0x8a, 0x8a, 0x8a, 0xff, 0x00, 0x00, 0x00, 0x90, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x99, 0x99, 0x99, 0x00,
        0x04, 0x04, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3e, 0x3e, 0x3e, 0xcb, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfe, 0xfe, 0xfe, 0xff, 0xff, 0xff, 0xff,
        0xff, 0xff, 0xff, 0xff, 0xff, 0xa2, 0xa2, 0xa2, 0xff, 0x13, 0x13, 0x13, 0xab, 0x00, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3f, 0x3f, 0x3f, 0xcb, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfe, 0xfe, 0xfe, 0xff,
        0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xb7, 0xb7, 0xb7, 0xff, 0x1f, 0x1f, 0x1f, 0xbb, 0x00, 0x00, 0x00, 0x17, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x41, 0x41, 0x41, 0xcc, 0xea, 0xea, 0xea, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
        0xfe, 0xfe, 0xfe, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xca, 0xca, 0xca, 0xff, 0x3d, 0x3d, 0x3d, 0xd8, 0x00, 0x00,
        0x00, 0x37, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x41, 0x41, 0x41, 0xcc, 0xea, 0xea, 0xea, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
        0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xde, 0xde, 0xde, 0xff, 0x56, 0x56, 0x56,
        0xef, 0x00, 0x00, 0x00, 0x4f, 0x00, 0x00, 0x00, 0x00,
    0x42, 0x42, 0x42, 0xcc, 0xea, 0xea, 0xea, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
        0xfe, 0xfe, 0xfe, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf3, 0xf3, 0xf3,
        0xff, 0x76, 0x76, 0x76, 0xff, 0x00, 0x00, 0x00, 0x6b,
    0x43, 0x43, 0x43, 0xcc, 0xea, 0xea, 0xea, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
        0xff, 0xff, 0xff, 0xff, 0xce, 0xce, 0xce, 0xff, 0x80, 0x80, 0x80, 0xf7, 0x74, 0x74, 0x74, 0xf8, 0x6d, 0x6d, 0x6d, 0xfb, 0x72,
        0x72, 0x72, 0xf8, 0x57, 0x57, 0x57, 0xff, 0x0c, 0x0c, 0x0c, 0xb3,
    0x44, 0x44, 0x44, 0xcc, 0xeb, 0xeb, 0xeb, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfc, 0xfb, 0xfb, 0xfb, 0xff, 0xfe, 0xfe, 0xfe,
        0xff, 0xff, 0xff, 0xff, 0xff, 0xc9, 0xc9, 0xc9, 0xff, 0x13, 0x13, 0x13, 0xb7, 0x00, 0x00, 0x00, 0x1b, 0x00, 0x00, 0x00, 0x14,
        0x00, 0x00, 0x00, 0x13, 0x00, 0x00, 0x00, 0x0c, 0x29, 0x29, 0x29, 0x07,
    0x45, 0x45, 0x45, 0xcc, 0xe8, 0xe8, 0xe8, 0xff, 0xff, 0xff, 0xff, 0xff, 0xd9, 0xd9, 0xd9, 0xff, 0x5e, 0x5e, 0x5e, 0xff, 0xe2, 0xe2,
        0xe2, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x62, 0x62, 0x62, 0xf0, 0x00, 0x00, 0x00, 0x13, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x45, 0x45, 0x45, 0xcc, 0xf9, 0xf9, 0xf9, 0xff, 0xec, 0xec, 0xec, 0xff, 0x4a, 0x4a, 0x4a, 0xd8, 0x00, 0x00, 0x00, 0x78, 0x8a, 0x8a,
        0x8a, 0xfb, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xc3, 0xc3, 0xc3, 0xff, 0x00, 0x00, 0x00, 0x8b, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x58, 0x58, 0x58, 0xd3, 0xd9, 0xd9, 0xd9, 0xff, 0x5e, 0x5e, 0x5e, 0xef, 0x00, 0x00, 0x00, 0x34, 0x00, 0x00, 0x00, 0x00, 0x3b,
        0x3b, 0x3b, 0xc7, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf4, 0xf4, 0xf4, 0xff, 0x54, 0x54, 0x54, 0xdc, 0x00, 0x00,
        0x00, 0x14, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0x00, 0x00, 0x00, 0x00,
    0x3e, 0x3e, 0x3e, 0xe0, 0x54, 0x54, 0x54, 0xff, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x57, 0x8e, 0x8e, 0x8e, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xb0, 0xb0, 0xb0, 0xff, 0x00, 0x00, 0x00,
        0x78, 0x00, 0x00, 0x00, 0x00, 0x0c, 0x0c, 0x0c, 0x04, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x68, 0x00, 0x00, 0x00, 0x00, 0x4f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x10, 0x4c, 0x4c, 0x4c, 0xd0, 0xec, 0xec, 0xec, 0xff, 0xff, 0xff, 0xff, 0xfc, 0xf4, 0xf4, 0xf4, 0xff, 0x53, 0x53,
        0x53, 0xd8, 0x00, 0x00, 0x00, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1e, 0x1e, 0x1e, 0x00, 0x04, 0x04, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x6f, 0xab, 0xab, 0xab, 0xff, 0xf6, 0xf6, 0xf6, 0xff, 0x80, 0x80, 0x80, 0xff, 0x31, 0x31,
        0x31, 0xac, 0x00, 0x00, 0x00, 0x17, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x09, 0x09, 0x09, 0x03, 0x02, 0x02, 0x02, 0x03, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1f, 0x2e, 0x2e, 0x2e, 0xd7, 0x38, 0x38, 0x38, 0xc7, 0x00, 0x00, 0x00, 0x47, 0x00,
        0x00, 0x00, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
}
```

Definition at line 15 of file mouse_cursor_icon.c.

6.26 mouse_cursor_icon.c

Go to the documentation of this file.

```
00001
00013 #include "lvgl/lvgl.h"
00014
00015 const uint8_t mouse_cursor_icon_map[] = {
```

```
    0x19, 0x19, 0x19, 0xb8, 0x1e, 0x1e, 0x1e, 0xc8, 0x00, 0x00, 0x00, 0x13, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x48, 0x48, 0x48, 0xcc, 0xb2, 0xb2, 0xb2, 0xff, 0x3a, 0x3a, 0x3a, 0xcc, 0x00, 0x00, 0x00, 0x24, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x0a, 0x0a, 0x0a, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3b, 0x3b, 0x3b, 0xc8, 0xf6, 0xf6, 0xf6, 0xff, 0xdc, 0xdc, 0xdc, 0xff, 0x43, 0x43, 0x43, 0xe0, 0x00, 0x00, 0x00, 0x3b,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x0a, 0x0a, 0x0a, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3b, 0x3b, 0x3b, 0xcb, 0xe6, 0xe6, 0xe6, 0xff, 0xff, 0xff, 0xff, 0xfc, 0xe5, 0xe5, 0xe5, 0xff, 0x59, 0x59, 0x59, 0xf3, 0x00,
    0x00, 0x00, 0x4f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3c, 0x3c, 0x3c, 0xcb, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf5, 0xf5, 0xf5, 0xff, 0x72, 0x72,
    0x72, 0xff, 0x00, 0x00, 0x00, 0x73, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3d, 0x3d, 0x3d, 0xcb, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0x8a, 0x8a, 0x8a, 0xff, 0x00, 0x00, 0x00, 0x90, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x99, 0x99, 0x99,
    0x00, 0x04, 0x04, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3e, 0x3e, 0x3e, 0xcb, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfe, 0xfe, 0xfe, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xa2, 0xa2, 0xa2, 0xff, 0x13, 0x13, 0x13, 0xab, 0x00, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x3f, 0x3f, 0x3f, 0xcb, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfe, 0xfe,
    0xfe, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xb7, 0xb7, 0xb7, 0xb7, 0xff, 0x1f, 0x1f, 0x1f, 0xbb, 0x00, 0x00, 0x00, 0x17,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x41, 0x41, 0x41, 0xcc, 0xea, 0xea, 0xea, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xfe, 0xfe, 0xfe, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xca, 0xca, 0xca, 0xff, 0x3d, 0x3d, 0x3d, 0xd8,
    0x00, 0x00, 0x00, 0x37, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x41, 0x41, 0x41, 0xcc, 0xea, 0xea, 0xea, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xde, 0xde, 0xde, 0xff, 0x56,
    0x56, 0x56, 0xef, 0x00, 0x00, 0x00, 0x4f, 0x00, 0x00, 0x00, 0x00,
    0x42, 0x42, 0x42, 0xcc, 0xea, 0xea, 0xea, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xfe, 0xfe, 0xfe, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf3, 0xf3,
    0xf3, 0xff, 0x76, 0x76, 0x76, 0xff, 0x00, 0x00, 0x00, 0x6b,
    0x43, 0x43, 0x43, 0xcc, 0xea, 0xea, 0xea, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
    0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xce, 0xce, 0xce, 0xff, 0x80, 0x80, 0x80, 0xf7, 0x74, 0x74, 0x74, 0xf8, 0x6d, 0x6d, 0x6d, 0xfb,
    0x72, 0x72, 0x72, 0xf8, 0x57, 0x57, 0x57, 0xff, 0x0c, 0x0c, 0x0c, 0xb3,
    0x44, 0x44, 0x44, 0xcc, 0xeb, 0xeb, 0xeb, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfc, 0xfb, 0xfb, 0xfb, 0xff, 0xfe, 0xfe,
    0xfe, 0xff, 0xff, 0xff, 0xff, 0xff, 0xc9, 0xc9, 0xc9, 0xff, 0x13, 0x13, 0x13, 0xb7, 0x00, 0x00, 0x00, 0x1b, 0x00, 0x00, 0x00,
    0x14, 0x00, 0x00, 0x00, 0x13, 0x00, 0x00, 0x00, 0x0c, 0x29, 0x29, 0x29, 0x07,
    0x45, 0x45, 0x45, 0xcc, 0xe8, 0xe8, 0xe8, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xd9, 0xd9, 0xd9, 0xff, 0x5e, 0x5e, 0x5e, 0xff, 0xe2,
    0xe2, 0xe2, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x62, 0x62, 0x62, 0xf0, 0x00, 0x00, 0x00, 0x13, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x45, 0x45, 0x45, 0xcc, 0xf9, 0xf9, 0xf9, 0xff, 0xec, 0xec, 0xec, 0xff, 0x4a, 0x4a, 0x4a, 0xd8, 0x00, 0x00, 0x00, 0x78,
    0x8a, 0x8a, 0x8a, 0xfb, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xc3, 0xc3, 0xc3, 0xff, 0x00, 0x00, 0x00, 0x8b, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x58, 0x58, 0x58, 0xd3, 0xd9, 0xd9, 0xd9, 0xff, 0x5e, 0x5e, 0x5e, 0xef, 0x00, 0x00, 0x00, 0x34, 0x00, 0x00, 0x00, 0x00,
    0x3b, 0x3b, 0x3b, 0xc7, 0xe9, 0xe9, 0xe9, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf4, 0xf4, 0xf4, 0xff, 0x54, 0x54, 0x54, 0xdc, 0x00,
    0x00, 0x00, 0x14, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x03, 0x00, 0x00, 0x00, 0x00,
    0x3e, 0x3e, 0x3e, 0xe0, 0x54, 0x54, 0x54, 0xff, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x57, 0x8e, 0x8e, 0x8e, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xb0, 0xb0, 0xb0, 0xff, 0x00, 0x00,
    0x00, 0x78, 0x00, 0x00, 0x00, 0x00, 0x0c, 0x0c, 0x0c, 0x04, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x68, 0x00, 0x00, 0x00, 0x4f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x10, 0x4c, 0x4c, 0x4c, 0xd0, 0xec, 0xec, 0xec, 0xff, 0xff, 0xff, 0xff, 0xfc, 0xf4, 0xf4, 0xf4, 0xff, 0x53,
    0x53, 0x53, 0xd8, 0x00, 0x00, 0x00, 0x18, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1e, 0x1e, 0x1e, 0x00, 0x04, 0x04, 0x04, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x6f, 0xab, 0xab, 0xab, 0xab, 0xff, 0xf6, 0xf6, 0xf6, 0xff, 0x80, 0x80, 0x80, 0xff, 0x31,
    0x31, 0x31, 0xac, 0x00, 0x00, 0x00, 0x17, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x09, 0x09, 0x09, 0x03, 0x02, 0x02, 0x02, 0x03, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x1f, 0x2e, 0x2e, 0x2e, 0xd7, 0x38, 0x38, 0x38, 0xc7, 0x00, 0x00, 0x00, 0x47,
    0x00, 0x00, 0x00, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
};

lv_image_dsc_t mouse_cursor_icon = {
    .header.magic = LV_IMAGE_HEADER_MAGIC,
    .header.w = 14,
    .header.h = 20,
    .header.stride = 56,
    .data_size = 280 * 4,
    .header.cf = LV_COLOR_FORMAT_ARGB8888,
    .data = mouse_cursor_icon_map,
};
```

6.27 simulator_settings.h File Reference

global simulator settings

#include "lvgl/lvgl.h"

Data Structures

- struct simulator_settings_t

6.27.1 Detailed Description

global simulator settings

The simulator settings is a global variable defined in simulator_settings.c

Copyright (c) 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file simulator_settings.h.

6.28 simulator_settings.h

Go to the documentation of this file.

```
00001
00014
00015 #ifndef SIMULATOR_SETTINGS_H
00016 #define SIMULATOR_SETTINGS_H
00017
00018 #ifdef __cplusplus
00019 extern "C" {
00020 #endif
00021
00022 /*********************
00023 *      INCLUDES
00024 *********************/
00025 #include "lvgl/lvgl.h"
00026
00027 /*********************
00028 *      DEFINES
00029 *********************/
00030
00031 /**********************
00032 *      TYPEDEFS
00033 **********************/
00034
00035 typedef struct {
00036     uint32_t window_width;
00037     uint32_t window_height;
00038     lv_display_rotation_t rotation;
00039     bool maximize;
00040     bool fullscreen;
00041 } simulator_settings_t;
00042
00043 /**********************
00044 * GLOBAL PROTOTYPES
00045 **********************/
00046
00047 /**********************
00048 *      MACROS
00049 **********************/
00050
00051 #ifdef __cplusplus
00052 } /*extern "C"*/
00053 #endif
00054
00055 #endif /*SIMULATOR_SETTINGS_H*/
```

6.29 simulator_util.c File Reference

Utility functions Copyright (c) 2025 EDGEMTech Ltd.

#include <stdlib.h>
#include <stdio.h>
#include <stdarg.h>

Functions

- const char * getenv_default (const char *name, const char *default_val)
  Wrapper around getenv(3), allowing to set a default value.
- void die (const char *msg,...)
  Centralized exit point, called due to an error.

6.29.1 Detailed Description

Utility functions Copyright (c) 2025 EDGEMTech Ltd.

Based on the original file from the repo

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file simulator_util.c.

6.29.2 Function Documentation

6.29.2.1 die()

```
void die (
            const char * msg,
            ...)
```

Centralized exit point, called due to an error.

Parameters

msg	The message to display on stderr before killing the program
...	Values for the format string.

Definition at line 52 of file simulator_util.c.

Referenced by configure_simulator(), and main().

6.29.2.2 getenv_default()

```
const char * getenv_default (
            const char * name,
            const char * default_val)
```

Wrapper around getenv(3), allowing to set a default value.

Parameters

name	The name of the environment variable
dflt	The default value to set if the variable is not present.

Returns

default value or value of environment variable.

Definition at line 45 of file simulator_util.c.

6.30 simulator_util.c

Go to the documentation of this file.

```
00001
00012
00013 /*********************
00014 *      INCLUDES
00015 *********************/
00016
00017 #include <stdlib.h>
00018 #include <stdio.h>
00019 #include <stdarg.h>
00020
00021 /*********************
00022 *      DEFINES
00023 *********************/
00024
00025 /*********************
00026 *      TYPEDEFS
00027 *********************/
00028
00029 /*********************
00030 *  STATIC PROTOTYPES
00031 *********************/
00032
00033 /*********************
00034 *  STATIC VARIABLES
00035 *********************/
00036
00037 /*********************
00038 *      MACROS
00039 *********************/
00040
00041 /*********************
00042 *   GLOBAL FUNCTIONS
00043 *********************/
00044
00045 const char * getenv_default(const char * name, const char * default_val)
00046 {
00047     const char * value = getenv(name);
00048     return value ? value : default_val;
00049 }
00050
00051
00052 void die(const char * msg, ...)
00053 {
00054     va_list args;
00055
00056     va_start(args, msg);
00057     vfprintf(stderr, msg, args);
00058     va_end(args);
00059
00060     exit(EXIT_FAILURE);
00061
00062 }
00063
00064 /*********************
00065 *   STATIC FUNCTIONS
00066 *********************/
```

6.31 simulator_util.h File Reference

simulator_util.h - Header file for the utility functions used by the simulator

#include <stdarg.h>

Functions

- const char * getenv_default (const char *name, const char *default_val)
  Wrapper around getenv(3), allowing to set a default value.
- void die (const char *msg,...)
  Centralized exit point, called due to an error.

6.31.1 Detailed Description

simulator_util.h - Header file for the utility functions used by the simulator

Copyright (c) 2025 EDGEMTech Ltd.

Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)

Definition in file simulator_util.h.

6.31.2 Function Documentation

6.31.2.1 die()

```
void die (
            const char * msg,
            ...)
```

Centralized exit point, called due to an error.

Parameters

msg	The message to display on stderr before killing the program
...	Values for the format string.

Definition at line 52 of file simulator_util.c.

Referenced by configure_simulator(), and main().

6.31.2.2 getenv_default()

```
const char * getenv_default (
            const char * name,
            const char * default_val)
```


Wrapper around getenv(3), allowing to set a default value.

Parameters

name	The name of the environment variable
dflt	The default value to set if the variable is not present.

Returns

default value or value of environment variable.

Definition at line 45 of file simulator_util.c.

6.32 simulator_util.h

Go to the documentation of this file.

```
00001
00011
00012 #ifndef SIMULATOR_UTIL_H
00013 #define SIMULATOR_UTIL_H
00014
00015 #ifdef ___cplusplus
00016 extern "C" {
00017 #endif
00018
00019 /*********************
00020 *      INCLUDES
00021 *********************/
00022 #include <stdarg.h>
00023
00024
00025 /*********************
00026 * GLOBAL PROTOTYPES
00027 *********************/
00028
00035 const char * getenv_default(const char * name, const char * default_val);
00036
00037
00043 void die(const char * msg, ...);
00044
00045 /*********************
00046 *      DEFINES
00047 *********************/
00048
00049 /**********************
00050 *      TYPEDEFS
00051 **********************/
00052
00053 /**********************
00054 *      MACROS
00055 **********************/
00056
00057
00058 #ifdef ___cplusplus
00059 } /*extern "C"*/
00060 #endif
00061
00062 #endif /*SIMULATOR_UTIL_H*/
```

6.33 main.c File Reference

#include <unistd.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "lvgl/lvgl.h"
#include "ui/ui.h"
#include "ui/actions.h"
#include "ui/vars.h"
#include "app_runtime.h"
#include "src/lib/driver_backends.h"
#include "src/lib/simulator_util.h"
#include "src/lib/simulator_settings.h"

Macros

- #define DEFAULT_DISPLAY_W "800"
- #define DEFAULT_DISPLAY_H "480"

Functions

- static void configure_simulator (int argc, char **argv)

  Configure simulator.
- static void print_lvgl_version (void)

  Print LVGL version.
- static void print_usage (void)

  Print usage information.
- static void ui_tick_timer_cb (lv_timer_t *timer)
- static void ui_runtime_sync_timer_cb (lv_timer_t *timer)

  Synchronize runtime data with UI native variables.
- void lv_ui_example (void)
- int main (int argc, char **argv)

  Entry point.
- void action_my_action (lv_event_t *e)

Variables

- static char * selected_backend
- simulator_settings_t settings

6.33.1 Macro Definition Documentation

6.33.1.1 DEFAULT_DISPLAY_H

#define DEFAULT_DISPLAY_H "480"

Definition at line 35 of file main.c.

Referenced by configure_simulator().

6.33.1.2 DEFAULT_DISPLAY_W

```
#define DEFAULT_DISPLAY_W "800"
```

Definition at line 34 of file main.c.

Referenced by configure_simulator().

6.33.2 Function Documentation

6.33.2.1 action_my_action()

```
void action_my_action (
            lv_event_t * e)
```

Definition at line 267 of file main.c.

6.33.2.2 configure_simulator()

```
void configure_simulator (
            int argc,
            char ** argv)    [static]
```

Configure simulator.

Process the command-line arguments used to select backend and window options before the simulator enters the LVGL run loop.

Parameters

argc	the count of arguments in argv
argv	The arguments

Definition at line 116 of file main.c.

References DEFAULT_DISPLAY_H, DEFAULT_DISPLAY_W, die(), driver_backends_is_supported(), driver_backends_print_supported(), driver_backends_register(), print_lvgl_version(), print_usage(), selected_backend, and settings.

Referenced by main().

6.33.2.3 lv_ui_example()

```
void lv_ui_example (
            void )    [extern]
```

Definition at line 7 of file ui_example.c.

References value_changed_event_cb().

6.33.2.4 main()

Parameters

argc	the count of arguments in argv
argv	The arguments

Definition at line 206 of file main.c.

References app_runtime_init(), app_runtime_start(), app_runtime_stop(), configure_simulator(), die(), driver_backends_init_backend(), driver_backends_register(), driver_backends_run_loop(), selected_backend, settings, ui_runtime_sync_timer_cb(), and ui_tick_timer_cb().

6.33.2.5 print_lvgl_version()

void print_lvgl_version (
void) [static]

Print LVGL version.

Definition at line 60 of file main.c.

Referenced by configure_simulator().

6.33.2.6 print_usage()

void print_usage (
void) [static]

Print usage information.

Definition at line 72 of file main.c.

Referenced by configure_simulator().

6.33.2.7 ui_runtime_sync_timer_cb()

void ui_runtime_sync_timer_cb (
lv_timer_t * timer) [static]

Synchronize runtime data with UI native variables.

Read the latest application snapshot produced by the secondary C++ runtime thread and copy its values into the native variables consumed by the EEZ UI bindings from the LVGL thread.

Parameters

timer	The LVGL timer instance

Definition at line 97 of file main.c.

References app_runtime_get_snapshot(), app_snapshot_t::myvar1, and set_var_myvar1().

Referenced by main().

6.33.2.8 ui_tick_timer_cb()

void ui_tick_timer_cb (
lv_timer_t * timer) [static]

Definition at line 82 of file main.c.

Referenced by main().

6.33.3 Variable Documentation

6.33.3.1 selected_backend

char* selected_backend [static]

Definition at line 48 of file main.c.

Referenced by configure_simulator(), and main().

6.33.3.2 settings

simulator_settings_t settings [extern]

Definition at line 103 of file driver_backends.c.

Referenced by configure_simulator(), and main().

6.34 main.c

Go to the documentation of this file.

```
00001 /*****************************************************************
00002 *
00003 * main.c - LVGL simulator for GNU/Linux
00004 *
00005 * Based on the original file from the repository
00006 *
00007 *
00008 * To see how each driver is initialized check the
00009 * 'src/lib/display_backends' directory
00010 *
00011 *
00012 * Author: EDGEMTech Ltd, Erik Tagirov (erik.tagirov@edgemtech.ch)
00013 *
00014 *****************************************************************/
00015 #include <unistd.h>
00016 #include <pthread.h>
00017 #include <stdio.h>
00018 #include <stdlib.h>
00019 #include <string.h>
00020
00021 #include "lvgl/lvgl.h"
00022
00023 #include "ui/ui.h"
00024 #include "ui/actions.h"
00025 #include "ui/vars.h"
00026
00027 #include "app_runtime.h"
00028
00029
00030 #include "src/lib/driver_backends.h"
00031 #include "src/lib/simulator_util.h"
00032 #include "src/lib/simulator_settings.h"
```

```

#define DEFAULT_DISPLAY_W "800"
#define DEFAULT_DISPLAY_H "480"


/* Internal functions */
static void configure_simulator(int argc, char ** argv);
static void print_lvgl_version(void);
static void print_usage(void);
static void ui_tick_timer_cb(lv_timer_t * timer);
static void ui_runtime_sync_timer_cb(lv_timer_t * timer);

/* contains the name of the selected backend if user
 * has specified one on the command line */
static char * selected_backend;

/* Global simulator settings, defined in lv_linux_backend.c */
extern simulator_settings_t settings;


extern void lv_ui_example(void);


static void print_lvgl_version(void)
{
    fprintf(stdout, "%d.%d.%d-%s\n",
            LVGL_VERSION_MAJOR,
            LVGL_VERSION_MINOR,
            LVGL_VERSION_PATCH,
            LVGL_VERSION_INFO);
}

static void print_usage(void)
{
    fprintf(stdout,
            "\nlvglsim [-V] [-B] [-f] [-m] [-b backend_name] [-W window_width] [-H window_height] [-R rotation]\n\n");
    fprintf(stdout, "-V print LVGL version\n");
    fprintf(stdout, "-B list supported backends\n");
    fprintf(stdout, "-f fullscreen\n");
    fprintf(stdout, "-m maximize\n");
}

static void ui_tick_timer_cb(lv_timer_t * timer)
{
    LV_UNUSED(timer);
    ui_tick();
}

static void ui_runtime_sync_timer_cb(lv_timer_t * timer)
{
    app_snapshot_t snapshot;

    LV_UNUSED(timer);

    app_runtime_get_snapshot(&snapshot);
    set_var_myvar1(snapshot.myvar1);
}

static void configure_simulator(int argc, char ** argv)
{
    int opt = 0;

    selected_backend = NULL;
    driver_backends_register();

    const char * env_w = getenv("LV_SIM_WINDOW_WIDTH");
    const char * env_h = getenv("LV_SIM_WINDOW_HEIGHT");
    /* Default values */
    settings.window_width = atoi(env_w ? env_w : DEFAULT_DISPLAY_W);
    settings.window_height = atoi(env_h ? env_h : DEFAULT_DISPLAY_H);

    /* Parse the command-line options. */
    while((opt = getopt(argc, argv, "b:fmW:H:R:BVh")) != -1) {
        switch(opt) {
            case 'h':
                print_usage();
                exit(EXIT_SUCCESS);
                break;
            case 'V':
                print_lvgl_version();
                exit(EXIT_SUCCESS);
                break;
            case 'B':
                driver_backends_print_supported();
                exit(EXIT_SUCCESS);
                break;
```

```
            case 'b':
                if(driver_backends_is_supported(optarg) == 0) {
                    die("error no such backend: %s\n", optarg);
                }
                selected_backend = strdup(optarg);
                break;
            case 'f':
                settings.fullscreen = true;
                break;
            case 'm':
                settings.maximize = true;
                break;
            case 'W':
                settings.window_width = atoi(optarg);
                break;
            case 'H':
                settings.window_height = atoi(optarg);
                break;
            case 'R':
                switch(atoi(optarg)) {
                    case 0:
                        settings.rotation = LV_DISPLAY_ROTATION_0;
                        break;
                    case 90:
                        settings.rotation = LV_DISPLAY_ROTATION_90;
                        break;
                    case 180:
                        settings.rotation = LV_DISPLAY_ROTATION_180;
                        break;
                    case 270:
                        settings.rotation = LV_DISPLAY_ROTATION_270;
                        break;
                    default:
                        LV_LOG_WARN("Invalid rotation angle. Valid angles are {0, 90, 180, 270}");
                        break;
                }
                break;
            case ':':
                print_usage();
                die("Option -%c requires an argument.\n", optopt);
                break;
            case '?':
                print_usage();
                die("Unknown option -%c.\n", optopt);
        }
    }
}




int main(int argc, char ** argv)
{

    configure_simulator(argc, argv);

    /* Initialize LVGL. */
    lv_init();

    /* Initialize the configured default backend */
    driver_backends_register();


    /* Initialize the configured backend */
    if(driver_backends_init_backend(selected_backend) == -1) {
        die("Failed to initialize display backend");
    }

    if(settings.rotation) {
#if LV_USE_DRAW_NANOVG && LV_DRAW_TRANSFORM_USE_MATRIX
        lv_display_set_matrix_rotation(NULL, true);
#endif
        lv_display_set_rotation(NULL, settings.rotation);
    }

    /* Enable for EVDEV support */
#if LV_USE_EVDEV
    // if(driver_backends_init_backend("EVDEV") == -1) {
    //     die("Failed to initialize evdev");
    // }
#endif

    /*Create a UI using .c file*/
    // lv_ui_example();

```

```
    printf("UI INIT !!!\n");
    fflush(stdout);
    /*Create a UI using generated files by eez-studio*/
    ui_init();

    if(app_runtime_init() != 0) {
        die("Failed to initialize app runtime");
    }

    if(app_runtime_start() != 0) {
        die("Failed to start app runtime");
    }

    /* Let LVGL drive the generated UI tick inside the active backend loop. */
    lv_timer_create(ui_tick_timer_cb, 10, NULL);
    lv_timer_create(ui_runtime_sync_timer_cb, 50, NULL);

    /* Enter the run loop of the selected backend */
    driver_backends_run_loop();

    app_runtime_stop();

    return 0;
}


void action_my_action(lv_event_t *e) {
    printf("Native action called !!!\n");
}
```

6.35 ui_example.c File Reference

#include "lvgl/lvgl.h"

Functions

- static void value_changed_event_cb (lv_event_t *e)
- void lv_ui_example (void)

6.35.1 Function Documentation

6.35.1.1 lv_ui_example()

void lv_ui_example (
void)

Definition at line 7 of file ui_example.c.

References value_changed_event_cb().

6.35.1.2 value_changed_event_cb()

void value_changed_event_cb (
lv_event_t * e) [static]

Definition at line 24 of file ui_example.c.

Referenced by lv_ui_example().

6.36 ui_example.c

Go to the documentation of this file.

```
#include "lvgl/lvgl.h"


static void value_changed_event_cb(lv_event_t * e);

void lv_ui_example(void)
{
    lv_obj_t * label = lv_label_create(lv_screen_active());

    /*Create an Arc*/
    lv_obj_t * arc = lv_arc_create(lv_screen_active());
    lv_obj_set_size(arc, 150, 150);
    lv_arc_set_rotation(arc, 135);
    lv_arc_set_bg_angles(arc, 0, 270);
    lv_arc_set_value(arc, 10);
    lv_obj_center(arc);
    lv_obj_add_event_cb(arc, value_changed_event_cb, LV_EVENT_VALUE_CHANGED, label);

    /*Manually update the label for the first time*/
    lv_obj_send_event(arc, LV_EVENT_VALUE_CHANGED, NULL);
}

static void value_changed_event_cb(lv_event_t * e)
{
    lv_obj_t * arc = lv_event_get_target_obj(e);
    lv_obj_t * label = (lv_obj_t *)lv_event_get_user_data(e);

    lv_label_set_text_fmt(label, "%" LV_PRId32 "%%", lv_arc_get_value(arc));

    /*Rotate the label to the current position of the arc*/
    lv_arc_rotate_obj_to_angle(arc, label, 25);
}

```

6.37 ui_logic.c File Reference

#include "ui/vars.h"

Functions

- int32_t get_var_myvar1 ()
- void set_var_myvar1 (int32_t value)

Variables

- int32_t myvar1

6.37.1 Function Documentation

6.37.1.1 get_var_myvar1()

int32_t get_var_myvar1 ()

Definition at line 5 of file ui_logic.c.

References myvar1.

6.37.1.2 set_var_myvar1()

```
void set_var_myvar1 (
            int32_t value)
```

Definition at line 9 of file ui_logic.c.

References myvar1.

Referenced by ui_runtime_sync_timer_cb().

6.37.2 Variable Documentation

6.37.2.1 myvar1

```
int32_t myvar1
```

Definition at line 3 of file ui_logic.c.

Referenced by get_var_myvar1(), and set_var_myvar1().

6.38 ui_logic.c

Go to the documentation of this file.

```
00001 #include "ui/vars.h"
00002
00003 int32_t myvar1;
00004
00005 int32_t get_var_myvar1() {
00006     return myvar1;
00007 }
00008
00009 void set_var_myvar1(int32_t value) {
00010     myvar1 = value;
00011 }
```